The Disciplined Agile Process Decision Framework

Scott W. Ambler[1](✉) and Mark Lines[2]

[1] Toronto, Canada
scott@scottambler.com
[2] Calgary, Canada
mark@scottambler.com

Abstract. The Disciplined Agile 2.0 process decision framework [1] provides light-weight guidance to help organizations streamline their information technology (IT) processes in a context-sensitive manner. It does this by showing how various activities such as solution delivery, operations, enterprise architecture, portfolio management, and many others work together in a cohesive whole. The framework also describes what these activities should address, provides a range of options for doing so, and describes the tradeoffs associated with each option. Every person, every team, and every organization is unique, therefore process frameworks must provide choices, not prescribe answers.

Keywords: Agile · Disciplined · Kanban · Scrum · Agility at scale · DevOps · Information technology department

1 History

To date there have been three major release tiers of this framework:

1. **Disciplined Agile Delivery 0.x.** The framework was originally developed at IBM Rational from early 2009 to June 2012. The IBM team worked closely with business partners, including Mark Lines, and was led by Scott Ambler. IBM Rational Method Composer (RMC) currently supports an early, 0.5 version of the DA framework.
2. **Disciplined Agile Delivery 1.x.** The DA 1.0 release occurred in June 2012 with publication of the first DA book, Disciplined Agile Delivery [2]. Evolution and publication of the DA framework continued at the Disciplined Agile site starting in August 2012. Ownership of the DA framework intellectual property effectively passed over to the Disciplined Agile Consortium [3] in October 2012, a fact that was legally recognized by IBM in June 2014. The focus was on the software delivery process.
3. **Disciplined Agile 2.x.** This is the current version of the framework, initially released in August 2015. The focus is on describing a flexible, context-sensitive approach to the entire IT process.

© Springer International Publishing Switzerland 2016
D. Winkler et al. (Eds.): SWQD 2016, LNBIP 238, pp. 3–14, 2016.
DOI: 10.1007/978-3-319-27033-3_1

2 Why Disciplined Agile?

There are several reasons why an organization should consider adopting the Disciplined Agile framework:

4. **Enable Agile Delivery Teams to Succeed.** The focus of Disciplined Agile Delivery (DA) 1.x [4] was tactical scaling of agile software development strategies across the delivery lifecycle in the range of situations that delivery teams find themselves in. The DA 1.x framework described how agile/lean teams work from beginning to end, showing how all the activities of solution delivery (analysis, design, testing, architecture, management, programming, and so on) fit together in a cohesive, streamlined whole. However, to succeed delivery teams must often work with people outside of the team, such as enterprise architects, operations engineers, governance people, data management people, and many others. For agile/lean delivery teams to be effective these people must also work in an agile/lean manner.
5. **Provide a Coherent Strategy for Agile IT.** The focus of Disciplined Agile 2.x is on strategic scaling of agile and lean strategies across the IT department. IT departments are complex adaptive organizations. What we mean by that is that the actions of one team will affect the actions of another team, and so on and so on. For example, the way that your agile delivery team works will have an effect on, and be affected by, any other team that you interact with. If you're working with your operations teams, perhaps as part of your overall Disciplined DevOps [5] strategy, then each of those teams will need to adapt the way they work to collaborate effectively with one another. Each team will hopefully learn from the other and improve the way that they work. These improvements with ripple out to other teams. The challenge is that every area within IT has one or more bodies of knowledge, and in some cases published "books of knowledge", that provide guidance for people working in those areas. These industry groups and their corresponding bodies of knowledge contradict one another, they are at different points on the agile/lean learning curve, and sometimes they promote very non-agile/lean strategies. At the IT level this can be very confusing, resulting in dysfunction. As you can see in Fig. 1, the DA framework shows how this all fits together in a flexible manner that supports the realities faced in complex adaptive systems.
6. **Support the Lean Enterprise.** A lean enterprise [6] is able to anticipate and respond swiftly to changes in the marketplace. It does this through an organizational culture and structure that facilitates change within the context of the situation that it faces. Lean enterprises require a learning mindset in the mainstream business and underlying lean and agile processes to drive innovation. This includes an IT department that is able to work in an agile/lean manner.
7. **Context Counts.** Every person, every team, and every organization is unique. The implication is that you need a framework that provides you with choices so that you can tailor, and later evolve, an approach to address the situation that you face in practice. Although prescriptive, one-size-fits-all frameworks such as SAFe [7] or Nexus [8] may seem like an attractive solution to your process-related needs at first, the reality is that they often do more harm than good within the organizations that adopt them.

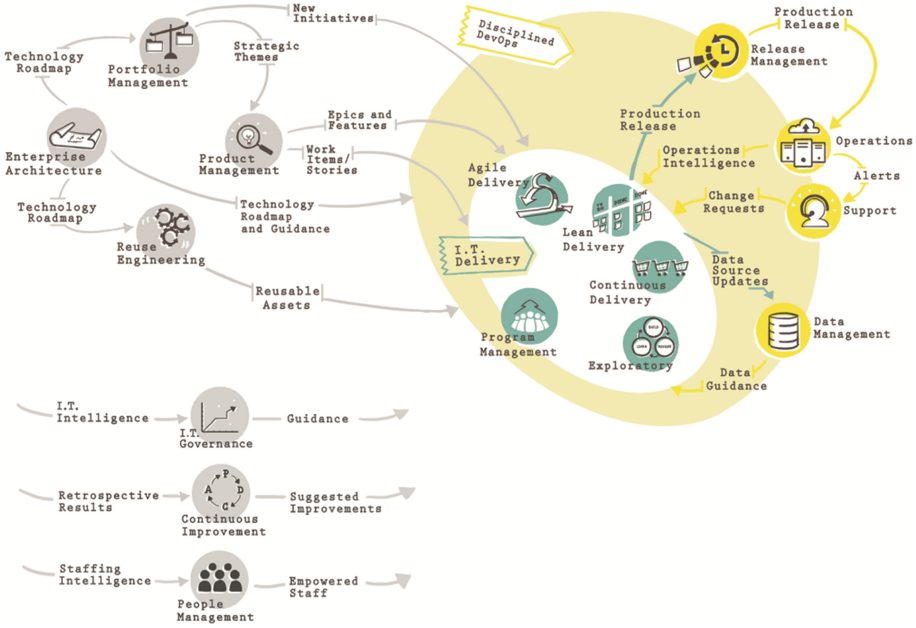

Fig. 1. The Disciplined Agile 2.0 Framework.

3 Tactical Agility at Scale

Many organizations start their agile journey by adopting Scrum because it describes a good strategy for leading agile software teams. However, Scrum is only a small part of what is required to deliver sophisticated solutions to your stakeholders. Invariably, teams need to look to other methods to fill in the process gaps that Scrum purposely ignores. When looking at other methods, there is considerable overlap and conflicting terminology that can be confusing to practitioners as well as outside stakeholders. Worse yet, people don't always know where to look for advice or even know what issues they need to consider. Then to compound the issue many teams find themselves in situations, such as geographically distributed teams or regulatory compliance, which "pure agilists" prefer to ignore.

To address these challenges, the Disciplined Agile (DA) process decision framework provides a more cohesive approach to agile solution delivery. It does this via several strategies.

Strategy #1: DA is a hybrid framework that builds upon the solid foundation of other methods and software process frameworks. One of the great advantages of agile and lean software development is the wealth of practices, techniques and strategies available to you. This is also one of its greatest challenges because without something like the DA framework, it's difficult to know what to choose and how to fit them together. Worse yet, many teams new to agile will adopt a method like Scrum or SAFe as if it's a recipe, ignoring advice from other sources and thereby getting into trouble. The DA framework

adopts practices and strategies from existing sources and provides advice for when and how to apply them together. In one sense, methods such as Scrum, Extreme Programming (XP), Kanban, and Agile Modeling (AM) provide the process bricks and DA the mortar to fit the bricks together effectively.

Strategy #2: DA supports a full delivery lifecycle. Figure 2 depicts a high-level view of the system lifecycle. The full system/product lifecycle goes from the initial concept for the product, through delivery, to operations and support and finally to retirement (not shown). The inner three phases – Inception, Construction, and Transition – form the delivery portion of the lifecycle. During this portion you incrementally build a consumable solution over time. Most systems will go through the delivery lifecycle many times.

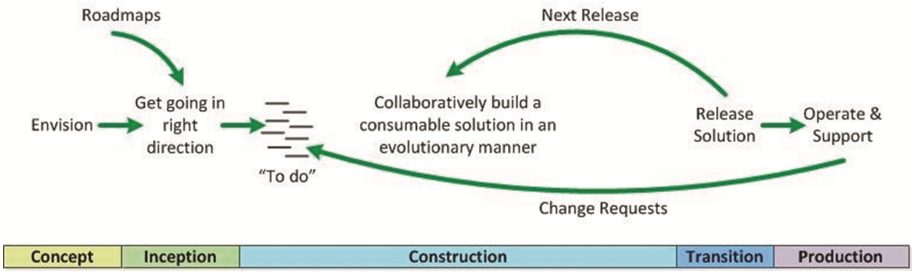

Fig. 2. A high-level view of the system lifecycle.

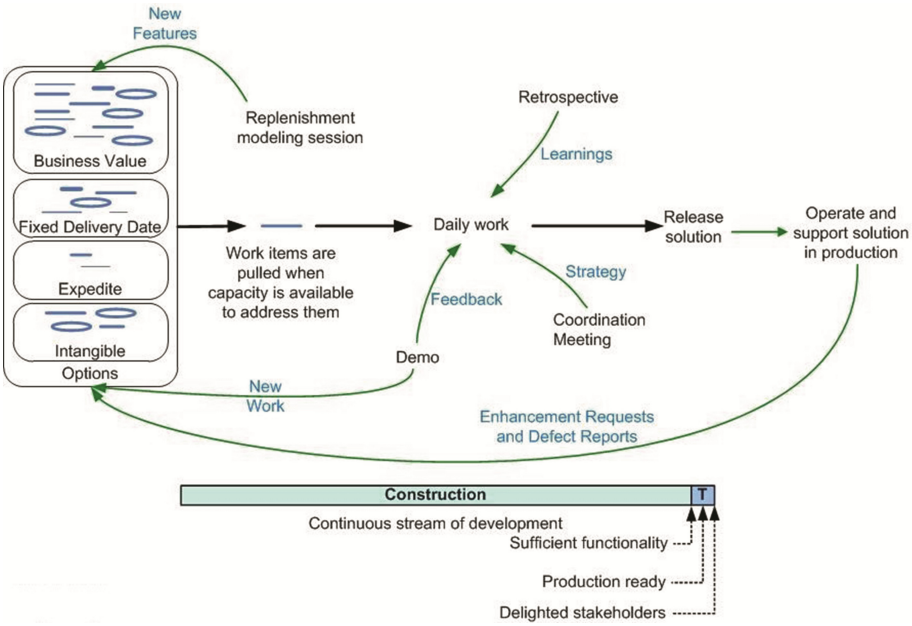

Fig. 3. Disciplined Agile's Continuous Delivery lifecycle.

Strategy #3: DA supports four delivery lifecycles. Because DA is not prescriptive and strives to reflect reality as best it can, it supports four versions of a delivery lifecycle [9]. These lifecycles are: an agile/basic version that extends the Scrum Construction lifecycle; an advanced/lean lifecycle based on Kanban; a continuous delivery lifecycle (shown in Fig. 3); and an exploratory lifecycle based upon a Lean Start-up approach [10]. DA teams will adopt a lifecycle that is most appropriate for their situation and then tailor it appropriately.

Strategy #4: DA is goal-driven, not prescriptive. The DA framework came about from empirical observations of dozens of teams apply agile and lean strategies in dozens of organizations working in different domains around the world. Although there were similarities between how these teams worked every team worked in a unique manner, and every team had spent considerable time and effort determining how to do so. And every team still had improvements to make, and many were struggling with doing so because they didn't have the process background to identify candidate options. It was clear to us that agile/lean teams could benefit from light-weight process guidance that described the range of options available to them.

To do this DA adopted a goal-driven, or capability-based, approach. The delivery goals are summarized in the mind map of Fig. 4. For example, the diagram indicates that when a team is in Inception that they must address goals such as forming the initial team, aligning with the enterprise direction (e.g. follow your corporate roadmaps and guidelines), develop an initial release plan, and explore the initial scope amongst other things. Although every team addresses these goals in some way, every team does so in a different manner and are likely to evolve their approach over time as they learn from experience.

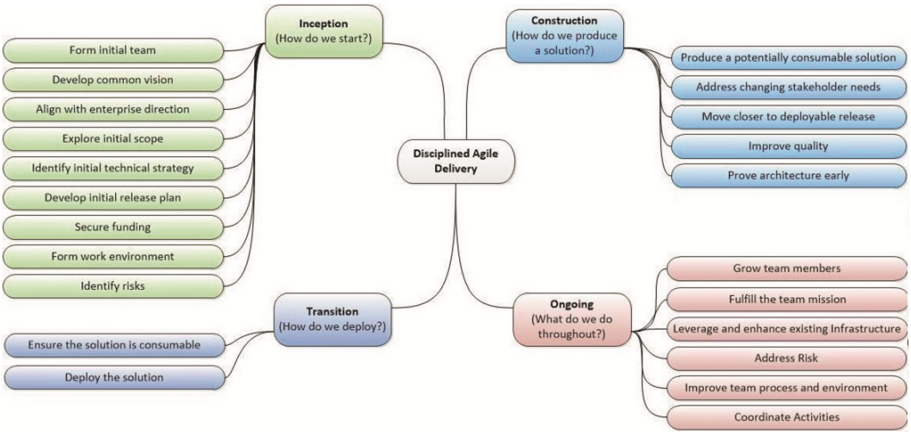

Fig. 4. Delivery goals (capabilities).

To provide teams more detailed guidance as to the process choices they have to address each goal the DA framework introduced the concept of process goal diagrams [11]. A process goal diagram depicts the process factors that should be considered when

addressing the goal and then a representative list of options for those goals. Because people around the world are constantly improving upon and identify new practices and strategies the list of options presented in the goal diagrams cannot possibly be definitive. Instead they represent the range of options available, letting people know that choices do exist (regardless of what prescriptive methodologies imply) and that sometimes some choices are distinctly better than others (again, regardless of what prescriptive methodologies imply).

Let's work through an example. Figure 5 depicts the goal diagram for Explore Initial Scope, a goal that you should address at the beginning of a project during the Inception phase. Where some agile methods will simply advise you to populate your product backlog with some initial user stories, the goal diagram makes it clear that you might want to be a bit more sophisticated in your approach. What level of detail should you capture, if any (a light specification approach of writing up some index cards and a few whiteboard sketches is just one option you should consider)? What view types should you consider (user stories are one approach to usage modeling, but shouldn't you consider other views to explore the data or the UI)? Default techniques, or perhaps more accurately suggested starting points, are shown in bold italics. Notice how we suggest that you likely want to default to capturing usage in some way, basic domain concepts (for example, via a high-level conceptual diagram) in some way, and non-functional requirements in some way. There are different strategies you may want to consider for modeling. You should also start thinking about your approach to managing your work. In DA, we make it clear that agile teams do more than just implement new requirements, hence our recommendation to default to a work item list over Scrum's simplistic

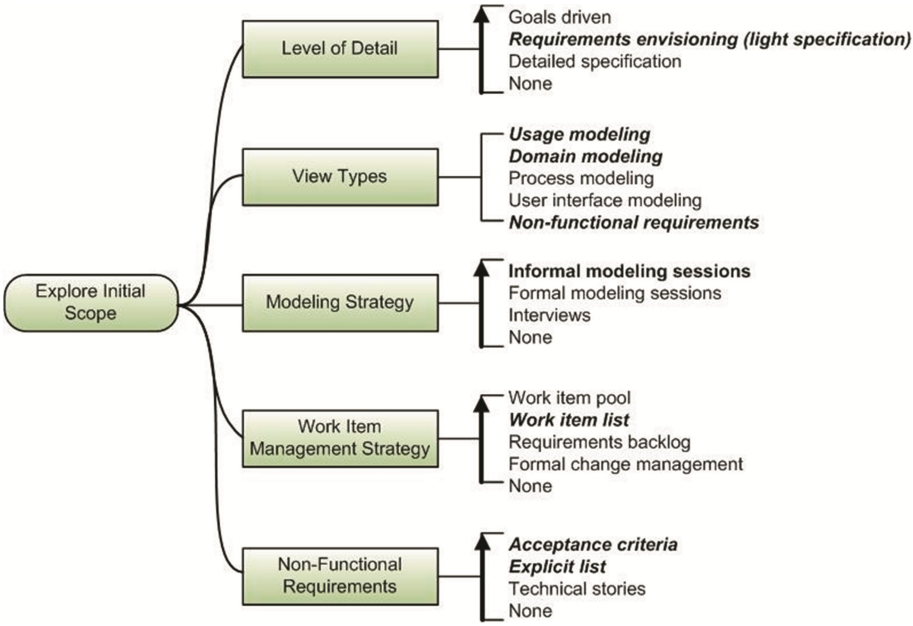

Fig. 5. The process goal diagram for Explore Initial Scope.

Requirements Backlog strategy. Work items may include new requirements to be implemented, defects to be fixed, training workshops, reviews of other teams' work, and so on. These are all things that need to be sized, prioritized, and planned for. Finally, the goal diagram makes it clear that when you're exploring the initial scope of your effort that you should capture non-functional requirements – such as reliability, availability, and security requirements (among many) – in some manner.

There are several fundamental advantages to taking a goal-driven approach to agile solution delivery. First, a goal-driven approach supports process tailoring by making process decisions explicit. Second, it enables effective scaling by guiding you through tailoring your strategy to reflect the realities of the scaling factors that you face. Third, it makes your process options very clear and thereby makes it easier to identify the appropriate strategy for the situation you find yourself in. Fourth, it takes the guesswork out of extending agile methods and thereby enables you to focus on your actual job, which is to provide value to your stakeholders. Fifth, it makes it clear what risks you're taking on and thus enables you to increase the likelihood of project success. Sixth, and this may not be a benefit, it hints at an agile maturity model.

Strategy #5: DA enables tactical scaling. When many people hear "scaling" they often think about large teams that may be geographically distributed in some way. This clearly happens, and people are clearly succeeding at applying agile in these sorts of situations, but there's often more to scaling than this. Organizations are also applying agile in compliance situations, either regulatory compliance that is imposed upon them or self-selected compliance (such as CMMI and ISO). They are also applying agile to a range

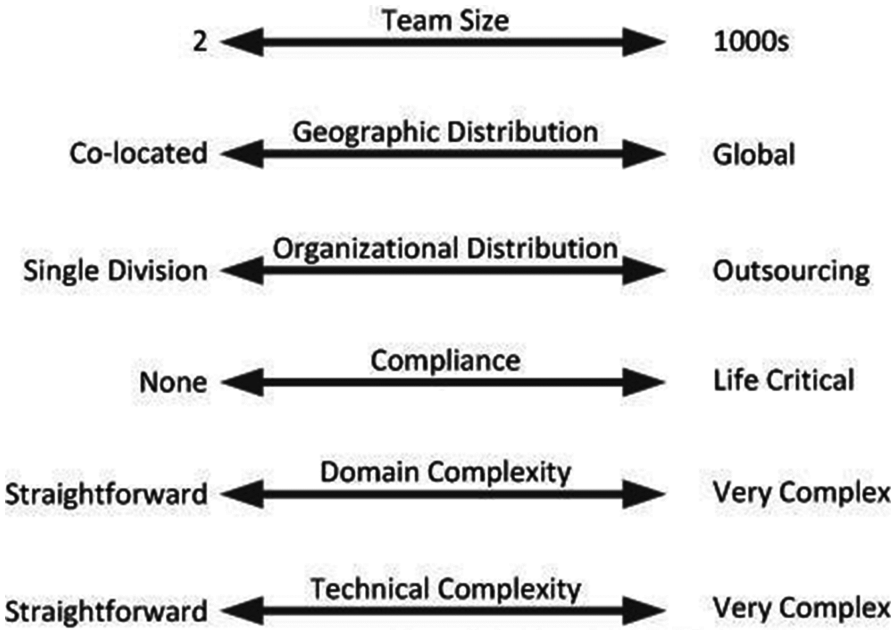

Fig. 6. Scaling factors faced by IT delivery teams.

of problem and solution complexities, and even when multiple organizations are involved (as in outsourcing). Figure 6 summarizes the potential scaling factors that you need to consider when tailoring your agile strategy [12].

Strategy #6: DA teams are enterprise aware. The observation is that DA teams work within your organization's enterprise ecosystem, as do all other teams. Often there are existing systems currently in production, and minimally your solution shouldn't impact them. Better yet, your solution will hopefully leverage existing functionality and data available in production. You will often have other teams working in parallel with your team and you may wish to take advantage of a portion of what they're doing and vice versa. Your organization may be working towards business or technical visions to which your team should contribute. A governance strategy exists which hopefully enhances what your team is doing.

Enterprise awareness is an important aspect of self-discipline because as a professional you should strive to do what's right for your organization and not just what's interesting to you. Delivery teams developing in isolation may choose to build something from scratch, or use different development tools, or create different data sources, when perfectly good ones that have been successfully installed, tested, configured, and fine-tuned already exist within the organization. Disciplined agile professionals will:

- Work closely with enterprise professionals, such as enterprise architects and portfolio managers
- Adopt and follow enterprise guidance
- Leverage enterprise assets, including existing systems and data sources
- Enhance your organizational ecosystem via refactoring enterprise assets
- Adopt a DevOps culture
- Share learnings and knowledge with other teams
- Adopt appropriate governance strategies, including open and honest monitoring.

4 Strategic Agility at Scale: The Disciplined Agile IT Department

A disciplined agile IT department is a flexible learning organization that is responsive to the needs of the organization(s) that it supports and is able to do so in a financially effective manner. DA 2.x extends disciplined agile strategies to the entire IT department. The development of DA 2.x began in the Spring of 2014 under the leadership of Scott and Mark. DA 2.x is based on several important observations. First, every organization is unique, and every IT department within each organization is also unique. Second, IT departments are dynamic complex adaptive systems that evolve over time. Third, the components of IT departments, teams and sub-departments, also evolve over time. Fourth, these components, when left to their own devices, are often not well aligned with each other or the enterprise. Worse yet, these groups may be working under their own locally optimized "improvement strategies." This misalignment is caused by competing leadership visions (or less delicately, by "politics") and exacerbated by disparate bodies of knowledge (BoKs) within our industry: The Agile Manifesto [13]; The Project Management Institute's BoK [14]; The Data Management BoK [15]; The Business Analysis BoK

[16]; The Open Group Architecture Framework (TOGAF) [17]; The Information Technology Infrastructure Library (ITIL) [18]; The Control Objectives for Information and Related Technology (COBIT) framework [19]; and many more.

Although all of these bodies of work provide valuable insight, they each provide their own locally optimized view of how things should work. These views overlap, they provide inconsistent advice, and they are often focused on a single specialty. For example, the BABoK provides a business analyst-centric view, TOGAF provides an architecture centric view, the DMBoK provides a data management centric view, and so on. All great views, but when combined with one another, which is a common approach in most organizations today looking for "best practices", they prove to be an ineffective mishmash. DA 2.x provides a coherent, integrated, high-level view of how an IT department may address all of these key areas in a consistent, flexible, and evolutionary manner. Wherever possible DA 2.x references the effective ideas in these BoKs and supplements them with strategies that are more consistent with modern agile approaches. Figure 1 overviewed the DA 2.x framework.

DA 2.x has been arranged into components called "process blades." Each process blade focuses on a major IT activity, as you can see in the previous diagram. No blade is an island unto itself – each one is involved in workflows with several other blades. The implication is that a change in one area, such as a process improvement or a change in the organizational structure of the people involved, will potentially affect the instantiation of the other blades. This interconnection of processes and organization strategies is a reflection of the fact that IT departments are complex adaptive systems. The process blades are:

- *Agile/Basic.* Describes the end-to-end solution delivery lifecycle for teams working in an agile, or Scrum-based, manner. Project teams who are new to agile, or who find themselves in situations where a regular work cadence is effective for them, will often choose to adopt this lifecycle.
- *Continuous Delivery.* Describes the end-to-end solution delivery lifecycle for teams working in a continuous delivery manner. Product teams who are working in a DevOps environment often adopt this strategy.
- *Continuous Improvement.* Addresses how to support process and organizational structure improvement across teams in a lightweight, collaborative manner; how to support improvement experiments within teams; and how to govern process improvement with the IT department.
- *Data Management.* Addresses how to improve data quality, evolve data assets such as master data and test data, and govern data activities within the organization.
- *Enterprise Architecture.* Addresses strategies for supporting stakeholders; supporting delivery teams; resolving technical dependencies between solutions; evolving the enterprise architecture; capturing the enterprise architecture; and governing the enterprise architecture efforts.
- *Exploratory/Lean Startup.* Describes the end-to-end solution delivery lifecycle for teams working in an exploratory, or "lean start up", manner. Teams who find themselves in situations where rapid innovation is called for often follow this lifecycle.
- *IT Governance.* Addresses strategies for consolidating various governance views, defining metrics, taking measurements, monitoring and reporting on measurements, developing and capturing guidance, defining roles and responsibilities, sharing

knowledge within the organization, managing IT risk, and coordinating the various governance efforts (including EA governance).

- *Lean/Advanced.* Describes the end-to-end solution delivery lifecycle for teams working in a lean, or Kanban-based, manner. Teams who have many small, relatively independent requirements (be they change requests or potential defects) and who are working on an existing solution will often adopt this lifecycle.
- *Operations.* Addresses how to run systems, evolve the IT infrastructure, manage change within the operational ecosystem, mitigate disasters, and govern IT operations.
- *Portfolio Management.* Addresses how to identify potential business value that could be supported by IT endeavors, explore those potential endeavors to understand them in greater detail, prioritize those potential endeavors, initiate the endeavors, manage vendors, and govern the IT portfolio.
- *Product Management.* Addresses strategies for managing a product, including allocating features to a product, evolving the business vision for a product, managing functional dependencies, and marketing the product line.
- *Program Management.* Addresses strategies for managing large product/project teams, allocating requirements between sub teams, managing dependencies between sub teams, coordinating the sub teams (via common or disparate cadences), and governing a program.
- *Release Management.* Addresses strategies for planning the IT release schedule, coordinating releases of solutions (such as release trains or release windows), managing the release infrastructure, supporting delivery teams, and governing the release management efforts.
- *Reuse Management.* Addresses how to identify and obtain reusable assets, publish the assets so that they are available to be reused, support delivery teams in reusing the assets, evolving those assets over time, and governing the reuse efforts.
- *Support.* Addresses how to adopt an IT support strategy, to escalate incidents, to effectively address the incidents, and govern the IT support effort.

The business environment is only becoming more competitive over time, with small nimble organizations competing in international marketplaces with large established competitors. This puts increasing pressure on existing enterprises to respond swiftly and effectively. They only way they can do this is if they have nimble IT departments that are sufficiently responsive. To increase the challenge, IT departments must be able to react to the changing needs of their organization while at the same time keep the existing IT infrastructure running smoothly. The only way that they can do this is by taking a flexible, holistic approach to the business of IT – This is exactly what Disciplined Agile 2.x is all about.

5 Principles for Effective Process Frameworks

The Disciplined Agile process decision framework is guided by the following principles:

- *Choice is good, and making informed choices is better.* Every team is a collection of unique individuals that face a unique situation within the context of a

unique organization. One process size does not fit all. To provide choice the DA framework supports four delivery lifecycles and is process goal driven. Most importantly the DA framework describes the tradeoffs involved with a myriad of agile and non-agile practices enabling people to make intelligent decisions regarding which practices to adopt given the current situation that they face.

- *Optimize the whole.* The DA framework addresses the full IT lifecycle, showing how it all fits together. Without an understanding of the larger process environment teams run the risk of locally optimizing their own processes to the detriment of the whole. For example, your data management team may have their own streamlined process based on traditional DAMA strategies, your delivery team may have their streamlined process based on the principles of the Agile Manifesto, and your operations team may have their streamlined process based on ITIL. Yet your overall process is ineffective because these three locally optimized strategies contradict and degrade one another when combined.
- *Every team owns its process.* Teams, and the individuals on them, must be free to improve the way that they work based on their learnings over time. In agile parlance we say that these teams "own their process".
- *Improve continuously.* Individuals, teams, and organizations must strive to continuously learn and improve the way that they work. The DA framework includes the process goal Improve Team Process and Environment which describes options for doing exactly what its name implies. It also has an explicit process blade Continuous Improvement that describes strategies for sharing improvements across teams, thereby speeding up your organization's process improvement efforts.
- *Embrace process change.* IT departments are complex adaptive systems. One implication of this is that any improvements that a team makes is that they change that the team works with other teams, motivating process improvements within those teams. Those changes will motivate improvements on other teams and so on. Disciplined agile teams are enterprise aware and understand that they will need to work with other teams to help them to understand and adopt new innovations, and be prepared to be helped by others to do the same.
- *Repeatable results are far more important than repeatable processes.* Effective teams focus on producing repeatable results, such as delivering high-quality software that meets stakeholder needs in a timely and cost effective manner. Because each team finds themselves in a unique situation, to be most efficient they need to follow a unique process tailored to reflect that situation. That "unique process" may be comprised of a relatively standard lifecycle and common practices such as architecture envisioning, database regression testing, non-solo development, and many others (granted, those practices may be tailored to reflect the situation too). Each team in your organization must be allowed to follow their version of the process, ideally sharing similar process components defined by a common process framework, to achieve the results required of them.
- *Empiricism is far more important than theory.* Observing how well a technique works in practice, and more importantly the context of the situations in which it (doesn't) work is far more valuable to practitioners than theories or prognostications about what should work. Theory has its place, but it is a poor cousin to empiricism.

The DA framework was originally developed based on observations of dozens of organizations worldwide, and has evolved since then based on learnings from many more. Furthermore it is backed up by our ongoing industry research [20].

IT departments are unique, complex adaptive systems. Anyone working in such environments needs a process framework that is sufficiently flexible to address the range of situations faced by your teams. The Disciplined Agile process decision framework is light-weight yet sufficiently flexible to support scaling at both the tactical and strategic levels.

References

1. The Disciplined Agile site. http://DisciplinedAgileDelivery.com
2. Ambler, S.W., Lines, M.: Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise. IBM Press, New York (2012)
3. Disciplined Agile Consortium. http://DisciplinedAgileConsortium.org
4. Introduction to Disciplined Agile Delivery (DAD) 1.x. http://DisciplinedAgileDelivery.com/introduction-to-disciplined-agile-delivery/
5. Disciplined DevOps. http://DisciplinedAgileDelivery.com/disciplineddevops/
6. Jumbler, J., Molesky, J., O'Reilly, B.: Lean Enterprise: How High-Performance Organizations Innovate at ScaleO'Reilly Media, Sebastopol (2015)
7. Scaled Agile Framework (SAFe). http://ScaledAgileFramework.com
8. The Scaled Scrum Framework. http://Scrum.org/Resources/What-is-Scaled-Scrum
9. Full Agile Delivery Lifecycles. http://DisciplinedAgileDelivery.com/lifecycle/
10. Ries, E.: The Lean Startup. How Today's Entrepenuers Use Innovation To Create Radically Successful Businesses. Crown Business, New York (2011)
11. Process Goals. http://DisciplinedAgileDelivery.com/process-goals/
12. Scaling Factors. http://DisciplinedAgileDelivery.com/agility-at-scale/scaling-factors/
13. The Agile Manifesto. http://AgileManifesto.org
14. PMBoK Guide and Standards. http://pmi.org/PMBOK-Guide-and-Standards.aspx
15. The Data Body of Knowledge. http://dama.org/content/body-knowledge
16. A Guide to the Business Body of Knowledge (BABoK). http://iiba.org/babok-guide.aspx
17. The Open Group Architecture Framework (TOGAF) v9.1. http://opengroup.org/togaf/
18. The Information Technology Infrastructure Library (ITIL). http://axelos.com/best-practice-solutions/itil
19. The Control Objectives for Information and Related Technology (COBIT) framework. http://isaca.org/COBIT/Pages/default.aspx
20. Surveys Exploring the Current State of Information Technology Practices. http://ambysoft.com/surveys/